ModKnapsack

Installation Guide for Linux

M. Myers

July 2026

Contents

1 What you are installing (the big picture) 2

2 Install the system build tools 3
2.1 Core packages (C compiler, Git, CMake, Python) 3
2.2 The Rust toolchain 3

3 Install OpenModelica 4

4 Get the ModKnapsack project 4

5 Build the Qiskit C API (inside the project) 5
5.1 Create and activate a Python environment 5
5.2 Download and compile Qiskit 5

6 Build the ModKnapsack C library 6
6.1 Tell the system where the Qiskit library lives 6

7 Verify everything works 6

8 Run the model through OpenModelica (the headline) 7
8.1 Choosing a test case and a solver path 7

9 Optional: Python analysis tools 7

10 Troubleshooting 8

11 Appendix 8
11.1 Validated tool versions 8
11.2 Directory layout after a successful install 9
11.3 Starting a fresh terminal later 9

This guide walks you, from a clean Linux machine, all the way to running the **ModKnapsack** quantum knapsack solver end-to-end through OpenModelica.

It assumes you:

- are on a **Debian/Ubuntu**-family Linux desktop (tested on **Ubuntu 24.04**).
- have **administrator (`sudo`) rights** – several steps install system software.
- have an internet connection and roughly **8–10 GB** of free disk space and **30–60 minutes** (most of it spent compiling Qiskit).
- are **new to OpenModelica, Qiskit, the Qiskit C API, and the shell** – so every command is given in full, with a short note on what it does and how to tell it worked.

No prior shell knowledge required. Open a *Terminal* (on Ubuntu: press the *Super/Windows* key, type `terminal`, press *Enter*). Then **copy each shaded command block and paste it in** (paste with *Ctrl+Shift+V*), pressing *Enter* after each block. When a command starts with `sudo`, the terminal will ask for your login password – type it (it stays invisible) and press *Enter*.

The project repository is `https://github.com/Legal-Ingenuity/2026ModelicaConference`; the clone command in Section 4 uses it directly.

How this guide differs from IBM's Qiskit documentation. These steps intentionally depart from IBM's official Qiskit installation documentation (`https://quantum.cloud.ibm.com/docs/en/guides/install-c-api`) for three reasons. (1) This guide is *project-specific*: it places the Qiskit C API exactly where the ModKnapsack build expects it (`qiskit/dist/c`), pins a validated Qiskit commit for reproducibility, and continues past Qiskit through the CMake build, OpenModelica, and running the model – none of which IBM's Qiskit-only document covers. (2) It *omits the separate `cbindgen` install* that IBM lists as a prerequisite: we verified that this version of Qiskit builds the C header with its own in-tree tool (which compiles `cbindgen` in as a library), so the standalone binary is never used. (3) It is written for readers new to these tools, so every command is spelled out.

1 What you are installing (the big picture)

Update, October 2026. This guide was written in July 2026 and the build steps are unchanged. Three things are new in the repository: (1) the model is a package with a library of twenty test instances and a `SolverPath` switch (Section 8.1); (2) `knapsack_test` exits non-zero on any failure, so the loop in Section 7 reports it; (3) the solver accepts up to seven items (seven-item runs take minutes). (4) `ModKnapsack.mo` finds the C source and the Qiskit library relative to its own folder (`modelica://` URIs), so nothing in it needs editing: keep `qiskit/` inside the project root as Section 4 and the Appendix describe.

ModKnapsack solves a small **0/1 knapsack problem** (pick the most valuable subset of items that fits in a cost budget) by building a **quantum circuit** and running **Grover's search** on a built-in simulator. The novelty is the *toolchain*: the whole thing is driven from an **OpenModelica** model.

```
OpenModelica model        C function            Qiskit C API          result
  ModKnapsack.mo   --->  callKnapsack()  --->  builds + runs   --->  back to
 (the .mo you run)      (knapsack/knapsack.c)  the quantum circuit    Modelica
```

So you will install **five** things, in the order of the sections that follow:

1. **System build tools** – a C compiler, Git, CMake, Python, and the Rust toolchain (Qiskit's C library is written in Rust).

2. **OpenModelica** – the modeling environment that runs `ModKnapsack.mo`.
3. **The ModKnapsack project** – this repository (the code, the model, this guide).
4. **The Qiskit C API** – cloned and compiled *inside* the project folder, producing the `libqiskit.so` library the C code links against.
5. **The ModKnapsack C library** – built with CMake — the library OpenModelica loads at run time.

Then you **verify** the build and **run the model**.

Two companion documents ship with the repository and pick up where this guide ends: `README.pdf` explains how the Modelica model calls the solver and what the three solver paths do, and `Knapsack_Documentation.pdf` is the full engineering reference. This guide gets you to a working install — those two explain what you have installed. (The accepted conference paper is in the repository as well.)

2 Install the system build tools

2.1 Core packages (C compiler, Git, CMake, Python)

Update the package list and install the essentials in one go:

```
sudo apt update
sudo apt install -y build-essential git cmake curl pkg-config \
                    python3 python3-venv python3-dev libssl-dev lsb-release
```

What this gives you:

`build-essential`
: the **GCC** C compiler and **make**.

`git`, `cmake`
: download the project and Qiskit, and build the C code.

`python3`, `python3-venv`, `python3-dev`
: Python, the tool to make an isolated "virtual environment", and the Python headers the Qiskit C library needs when it links.

`pkg-config`, `libssl-dev`
: build dependencies the Rust code commonly needs.

`curl`, `lsb-release`
: used by later steps.

2.2 The Rust toolchain

Qiskit's C library is compiled from Rust, so you need the Rust toolchain. (The build also generates the C header itself, using a library it compiles in automatically – you do *not* need to install `cbindgen` separately.) Install Rust with the official installer:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Press **Enter** to accept the default ("1) Proceed with standard installation"). When it finishes, load Rust into your **current** terminal:

```
source "$HOME/.cargo/env"
```

Verify the tools are present (each should print a version, not "command not found"):

```
gcc --version
cmake --version
git --version
python3 --version
rustc --version
cargo --version
```

3 Install OpenModelica

OpenModelica is installed from its official software repository. The commands below follow the **official instructions** at `https://openmodelica.org/download/download-linux/` – if that page differs from what's shown here, trust the page (repository URLs and keys occasionally change).

```
# 1. Import OpenModelica's signing key
curl -fsSL https://build.openmodelica.org/apt/openmodelica.asc \
  | sudo gpg --dearmor -o /usr/share/keyrings/openmodelica-keyring.gpg

# 2. Add the OpenModelica software repository (stable "release" channel)
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/openmodelica
    ↪ -keyring.gpg] https://build.openmodelica.org/apt $(lsb_release -cs) release" \
  | sudo tee /etc/apt/sources.list.d/openmodelica.list

# 3. Install OpenModelica
sudo apt update
sudo apt install -y openmodelica
```

Verify:

```
omc --version
```

You should see a version line such as `OpenModelica 1.27.0`. The command-line compiler `omc` is all this project needs (the graphical editor *OMEdit* is optional).

4 Get the ModKnapsack project

Pick a folder for your work (your home directory is fine), download the project, and move into it. This creates a folder named `ModKnapsack`:

```
cd ~
git clone https://github.com/Legal-Ingenuity/2026ModelicaConference.git ModKnapsack
cd ModKnapsack
```

Important: every remaining command in this guide is run **from inside this `ModKnapsack` folder** (your "project root"). If you open a new terminal later, return here first with `cd ~/ModKnapsack`.

You can confirm you're in the right place – these files should be listed:

```
ls ModKnapsack.mo CMakeLists.txt run_knapsack.mos knapsack/knapsack.c
```

If that `ls` reports "No such file or directory", the build files are in a **subfolder** of the repository. Run `ls` on its own to see the contents, `cd` into the folder that contains `CMakeLists.txt`, and treat **that** folder as your project root for the rest of this guide.

5 Build the Qiskit C API (inside the project)

The Qiskit C library is **not** part of the repository – you build it locally, *inside* the project folder, so the project's build files can find it at `qiskit/dist/c`.

5.1 Create and activate a Python environment

A "virtual environment" keeps these Python packages separate from the rest of your system. Create one named `.venv` and turn it on:

```
python3 -m venv .venv
source .venv/bin/activate
```

Your prompt now starts with `(.venv)`. (To turn it off later, type `deactivate`. If you open a new terminal, re-activate with `source .venv/bin/activate` from the project root.)

5.2 Download and compile Qiskit

Clone Qiskit **into the project folder** (this creates `ModKnapsack/qiskit`):

```
git clone https://github.com/Qiskit/qiskit.git
cd qiskit
```

Recommended for reproducibility. This project has been validated against Qiskit commit `61e3ca016` (Qiskit `2.5.0.dev0`; an earlier validated commit was `c5a8ecfd9`). A much newer Qiskit may change the C API and break the build. To pin the validated version:

```
git checkout 61e3ca016
```

(Skip this only if you intend to track the latest Qiskit.)

Build and install the Qiskit Python package (this compiles Rust and **can take 10–30 minutes** – it is normal for it to look busy):

```
pip install .
```

Now build the **C API** itself:

```
make c
```

Verify the two artifacts the project needs now exist:

```
ls dist/c/lib/libqiskit.so dist/c/include/qiskit.h
```

Both paths should be listed. Then go back to the project root:

```
cd ..
```

6 Build the ModKnapsack C library

From the project root, configure and build with CMake. This compiles the solver plus its self-test programs:

```
cmake -S . -B cmake-build-debug
cmake --build cmake-build-debug
```

This produces, in `cmake-build-debug/`, the shared library `libknapsack.so` (the one OpenModelica loads) and several test executables.

6.1 Tell the system where the Qiskit library lives

At run time, the programs need to find `libqiskit.so`. The cleanest way is to set one environment variable **per terminal**, from the project root:

```
export LD_LIBRARY_PATH="$PWD/qiskit/dist/c/lib:$LD_LIBRARY_PATH"
```

(If you open a fresh terminal later, run this again from the project root. If you ever see `error while loading shared libraries: libqiskit.so`, this is the line you forgot.)

7 Verify everything works

Run the test programs. Each prints its own results; the expected pass counts are shown in the comments. Run them from the project root (with `LD_LIBRARY_PATH` set as above):

```
./cmake-build-debug/gate_test       # 6/6   quantum-gate simulator
./cmake-build-debug/adder_test      # 23/23 reversible adder
./cmake-build-debug/cmp_test        # 19/19 comparator / feasibility flag
./cmake-build-debug/oracle_test     #       Grover oracle self-check
./cmake-build-debug/diffuser_test   #       Grover diffuser
./cmake-build-debug/solver_test     #       full solver (takes ~50 s -- be patient)
./cmake-build-debug/qasm_test       # 9/9   circuit export
./cmake-build-debug/draw_test       # 15/15 circuit drawing
./cmake-build-debug/knapsack_test   # 7/7   full end-to-end in C
```

If those pass, the quantum solver works. `solver_test` is randomized and runs for roughly **50 seconds** – it has not frozen.

8 Run the model through OpenModelica (the headline)

This is the whole point: OpenModelica drives the quantum solver. From the project root (with `LD_LIBRARY_PATH` still set):

```
omc run_knapsack.mos
```

`omc` loads `ModKnapsack.mo`, generates and compiles the simulation, calls `callKnapsack()` (which builds and runs the quantum circuit), and returns the answer to the model. For the built-in default problem instance the solver selects **items 0 and 2** (total **value 8** at a **total cost of 6**) and reports `ExStatus = 0` (success). The run should finish without any `getErrorString()` error text.

That's a complete install – OpenModelica → C → Qiskit C API → quantum search → result, end to end.

8.1 Choosing a test case and a solver path

`ModKnapsack.mo` holds a library `ModKnapsack.TestCases` of twenty instances: the four-item golden case above (`Golden_N4`), a two-item case used on IBM hardware (`HW_N2`), and three families at two to seven items — `TC1_N2` … `TC1_N7` (loose budget), `TC2_...` (tight budget), `TC3_...` (unique optimum). To run another case, put a two-line model in a file (say `run_tc3.mos`) and simulate it:

```
loadFile("ModKnapsack.mo"); getErrorString();
loadString("model Run extends ModKnapsack.KnapsackRun(tc = ModKnapsack.TestCases.TC3_N5
    ↪ , MaxCost = 4); end Run;");
simulate(Run, stopTime = 0.0); getErrorString();
```

In OMEdit the case is a drop-down on the `KnapsackRun` parameter dialog. `MaxCost` overrides the case's default budget. `SolverPath` selects the solver: **0** (default) runs the search in the built-in statevector simulator; **1** (Aer noise model) and **2** (IBM QPU) export the circuit as `solverCircuit.qasm` and then *wait up to two hours* for a results file (`aer_results.json` / `qpu_results.json`) that a separate program must write — the repository does not include that program, so leave `SolverPath` at 0 unless you have one (the format is described in `README.pdf`). The output of a run ends with the three lines the install gate reads (`ExStatus`, `PatentIndices`, `ActualCost`) and a table of the most probable subsets.

9 Optional: Python analysis tools

The repository also includes small Python helper scripts that are **not** needed for the steps above (the core C/OpenModelica path doesn't use Python at run time). They analyze or render the circuits the C code exports. To use them, activate the environment and add a few packages:

```
source .venv/bin/activate
pip install qiskit-aer qiskit-qasm3-import qiskit-ibm-runtime pylatexenc matplotlib
```

Script	What it does	Extra needs
`verify_qasm.py`	Round-trips exported circuits through Qiskit (fidelity)	—
`noise_study.py`	Simulates the circuit under device noise	`qiskit-aer`
`plot_noise.py`	Plots the noise-study results	`matplotlib`
`hardware_run.py`	Submits the circuit to a simulated or real IBM backend	IBM token (real QPU)
`draw_circuit.py`	Renders a circuit to a figure	a LaTeX install

For `draw_circuit.py` rendering, install LaTeX:

```
sudo apt install -y texlive-latex-extra texlive-fonts-recommended
```

10 Troubleshooting

Symptom	Cause and fix
`error while loading shared libraries: libqiskit.so`	`LD_LIBRARY_PATH` not set in this terminal. Re-run the `export LD_LIBRARY_PATH=...` line from Section 6.1 (from the project root).
CMake: `Could NOT find QISKIT_LIB`	`qiskit/dist/c/lib/libqiskit.so` doesn't exist – you haven't run `make c` yet, or it failed.
`make c` fails while linking Python	Install the Python headers: `sudo apt install -y python3-dev`, then retry.
Compile errors in `knapsack.c` against `qiskit.h`	You cloned a newer Qiskit whose C API changed. Pin the validated commit (`git checkout 61e3ca016` in `qiskit/`), re-run `make c`, then rebuild with CMake.
`omc` reports an error / can't load the model	Make sure you are in the project root, `ModKnapsack.mo` is present, and `LD_LIBRARY_PATH` is set.
`solver_test` seems stuck	It isn't – the randomized search runs about 50 seconds.

11 Appendix

11.1 Validated tool versions

This project was confirmed working with:

Tool	Version
OS	Ubuntu 24.04
OpenModelica	1.27.0 (dev.beta.1)
GCC	13.3.0
CMake	3.28.3
Python	3.12.3
Rust (`rustc`/`cargo`)	1.96.0
Qiskit	commit `61e3ca016` (`v2.5.0.dev0`)

Note: the build selects its own Rust toolchain. Qiskit's `rust-toolchain.toml` makes `rustup` fetch and use the version it pins (`1.87`, Rust edition 2024) to compile the C library, regardless of the host `rustc`; any current `rustup` installation works.

11.2 Directory layout after a successful install

```
ModKnapsack/                  project root - run all commands here
|-- ModKnapsack.mo            the OpenModelica model you run
|-- run_knapsack.mos          the script that loads + simulates it
|-- CMakeLists.txt            build configuration
|-- knapsack/                 the C source (knapsack.c + test programs)
|-- .venv/                    Python virtual environment (you created)
|-- qiskit/                   Qiskit clone (you created)
|   \-- dist/c/
|       |-- lib/libqiskit.so  the C library the build links against
|       \-- include/qiskit.h  its C header
\-- cmake-build-debug/        compiled output (you created)
    |-- libknapsack.so        loaded by OpenModelica
    \-- *_test                verification programs
```

11.3 Starting a fresh terminal later

If you come back after closing the terminal, three quick lines restore your working state from the project root:

```
cd ~/ModKnapsack
source .venv/bin/activate
export LD_LIBRARY_PATH="$PWD/qiskit/dist/c/lib:$LD_LIBRARY_PATH"
```

Disclosure

During the preparation of this work the author used Anthropic's Claude (LLM chosen appropriate to task and cost, via Claude Code) as a development and editing assistant for the underlying **ModKnapsack** project — to help resolve build and integration issues with the experimental Qiskit C API and to generate and debug test-harness scripts — and to draft and revise this installation guide from the validated install procedure. All steps were directed, independently

executed, and verified by the author, who retains sole intellectual property ownership of, and full responsibility for, the final work.